

ENTERPRISE ARCHITECTURE & ENGINEERING HANDBOOK

Indoore Backend API Automation Framework

A layered, standards-based approach to enterprise backend quality assurance for Indoore MDMS

Version	2.0 · July 2026
Classification	Confidential — Internal Engineering Use
Prepared by	Vamsi Krishna · QA Automation Engineer
Product ref	MDM Presentation_23.12.2025.pdf (131 pages)
Stack	Playwright · TypeScript · Node.js · PostgreSQL · Zod · GitHub Actions · Allure
Volumes	1–6 (Foundation · Core · Pillars · Modules · CI/CD · Delivery & MDM Alignment)

Indoore Backend API Automation Framework

Enterprise Architecture & Engineering Handbook

Field	Value
Document title	Indoore Backend API Automation Framework — Enterprise Handbook
Version	2.0
Date	July 2026
Classification	Confidential — Internal Engineering Use
Prepared by	Vamsi Krishna · QA Automation Engineer
Audience	QA, Backend Engineering, DevOps, Engineering Leadership
Supersedes	Handbook v1.0 (Volumes 1–5, July 2026)
Product reference	Indoore MDMS · MDM Presentation_23.12.2025.pdf (131 pages)

Core stack: Playwright · TypeScript · Node.js · PostgreSQL · Zod · GitHub Actions · Allure

Document control

Version history

Version	Date	Author	Description
1.0	July 2026	Vamsi Krishna	Initial five-volume enterprise handbook
2.0	July 2026	Vamsi Krishna	Professional refresh; delivery status; MDM PDF alignment; test POV accomplishments; current inventory metrics

Scope of this edition

This handbook consolidates **six volumes**:

- 1. Framework foundation
- 2. Core engineering components
- 3. Four quality pillars
- 4. Module architecture & catalogue
- 5. CI/CD & engineering operations
- 6. **Delivery status, MDM alignment & what we have achieved (test POV)** — *new in v2.0*

Companion living docs (keep in sync for day-to-day work):

Doc	Purpose
README.md	Setup & commands
CONTRIBUTING.md	PR standards
docs/ARCHITECTURE.md	Layered design
docs/MODULE_GUIDE.md	Add a module in ~30 minutes
docs/HARDENING-STATUS.md	Module × pillar depth
docs/MDM-PRESENTATION-COVERAGE.md	PDF slide → API follow/gap

Volume 1 — Framework Foundation

1. Executive summary

The **Indoore Backend API Automation Framework** is a layered, TypeScript-first platform that validates REST APIs for the Indoore **MDMS** (Meter Data Management System). It is implemented with **Playwright Test** as the runner and HTTP client, and is designed for enterprise maintainability rather than one-off scripts.

Unlike suites that only assert HTTP status and a few JSON fields, this framework validates **backend behaviour end-to-end** through:

- Strict **Zod** response contracts
- **Contract snapshots** that freeze intentional API shapes
- **Mutation-proof** tests that prove validators catch regressions
- Optional **read-only PostgreSQL** cross-checks (gated per module)
- Shared **ValidationEngine** / **AssertionEngine** and auth lifecycle

As of July 2026 inventory (`npm run test:inventory`):

Metric	Approximate count
Business modules	25
API-layer files	~140+
Spec files	~340+
Detected test cases	~900+ (inventory heuristic)
Contract snapshot packs	25 module folders under <code>contract-snapshots/</code>

Current phase: Framework **architecture is complete**. Work has shifted to **maintenance and expansion** — new APIs, pillar depth, CI reliability, documentation — not redesign of core layers.

2. Business problem

Before this framework matured, backend quality work faced:

- Manual API↔DB comparison
- Duplicated validation logic across modules
- Weak or inconsistent schema enforcement
- Silent API contract drift
- High maintenance cost
- Uneven module quality
- Limited confidence in regression packs

These issues scaled poorly as MDMS modules (billing, HES commands, revenue protection, MIS, etc.) grew.

3. Vision

A **single reusable automation platform** that:

- Supports all Indoore backend modules with one pattern
- Separates test defects from backend/infra findings
- Onboards a new engineer to a productive first module quickly
- Survives API evolution via contracts + mutation proof

4. Objectives

1. Standardize backend API automation across MDMS.
2. Minimize duplication via shared engines and module layers.
3. Raise regression confidence for QA → main promotions.
4. Enforce Zod schemas and contract snapshots.
5. Cross-validate critical data with read-only SQL where gated.
6. Prove validator strength with mutation-proof suites.
7. Enable ~30-minute module scaffolding for new APIs.
8. Align coverage with the MDM product presentation (API scope).

5. Technology stack

Category	Choice
Language	TypeScript (strict)
Test runner / HTTP	Playwright Test
Runtime	Node.js 20 (CI)
Schema	Zod
Database	PostgreSQL (pg) — read-only tests
Reporting	Playwright HTML/JSON + Allure
CI/CD	GitHub Actions (module gate + regression)
Package manager	npm

6. Architecture principles

- Separation of concerns (Api ≠ Validator ≠ Spec)
- Single responsibility per layer
- Reuse over copy-paste
- Strong typing
- Validation-first design
- Read-only DB access
- Soft-skips logged as **BACKEND FINDING** (tech debt, not “done”)

7. Layered design

Layer	Folder	Responsibility
Api	Api/	HTTP only — timing, typed envelope, no assertions
Data	Data/	Paths, payloads, env defaults
Mapper	Mapper/	Normalize raw JSON
Validator	Validator/	Business rules
Schema	schemas/	Zod contracts
Db	Db/	SQL + compare (gated)
Tests	tests/	Smoke, negative, contract, mutation, db, e2e

```
Spec → Api → Mapper → Validator / Zod → ValidationEngine → Report
      ↘ optional Db compare (gated)
```

8. Request lifecycle

1. `global.setup.ts` validates env and warms/reuses auth (`playwright/.auth/`).
2. Spec receives `authenticatedApi` fixture.
3. Api layer calls backend with Bearer/CSRF + auto-refresh on 401.
4. Mapper normalizes response.
5. Standard checks (status, content-type, time, sensitive data).
6. Domain validators + optional Zod.
7. Optional contract snapshot / DB compare / mutation (suite-dependent).
8. Finalize with defect context for triage.

9. Benefits by audience

Audience	Benefit
Developers	Early contract/regression signal
QA	Consistent module pattern; less manual DB checking
DevOps	Module-scoped CI gates
Leadership	Measurable coverage vs MDMS product areas

Volume 2 — Core Framework Components

Authentication engine

- Login: `POST /indore/auth/login` via `AuthApi`
- Token cache: `TokenManager` + `playwright/.auth/token.json`
- Requests: `authenticated.request.ts` with refresh retry
- Required env: `BASE_URL` , `EMAIL / USERNAME` , `PASSWORD`

Shared engines

Component	Role
<code>ValidationEngine</code>	Named checks + pass/fail summary
<code>AssertionEngine</code>	Status, content-type, timing, PII guards
<code>ApiValidationHelper</code>	Standard checks + finalize
<code>RetryEngine</code>	Transient HTTP retries
<code>BackendResponse</code>	5xx / 429 soft-handling patterns
Timeouts (<code>api-timeouts.ts</code>)	Module budgets (HES poll, billing, consumption, ...)

HES async specialty

HES command E2E posts jobs (`IN_PROGRESS`) and polls `query-meter-job` . Completion depends on HES callback. Default path validates async pending state; set `HES_E2E_REQUIRE_COMPLETION=true` for strict FINISHED.

Design patterns in use

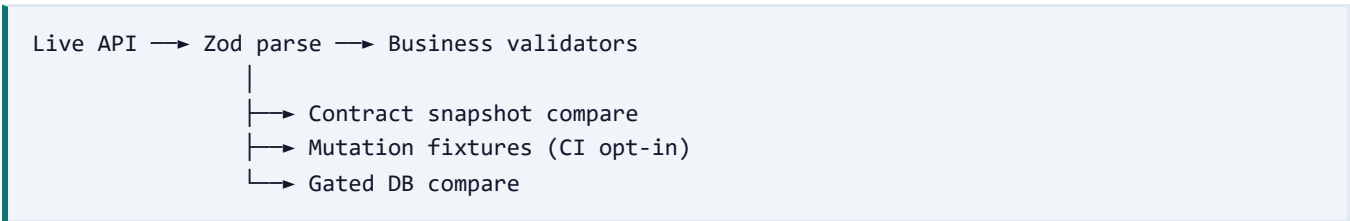
- Layered module template
- Fixture injection (`authenticatedApi` , `api+db`)
- Gate flags for expensive/env-bound suites (`*_DB_SQL_READY`)
- Scaffold script for four-pillar stubs

Volume 3 — Four Quality Pillars

Why four pillars?

HTTP assertions alone miss: shape drift, weak validators, and API≠DB divergence. The pillars address those failure modes deliberately.

Pillar	Mechanism	Tag / gate
1. Zod schemas	<code>.strict()</code> envelopes where possible	<code>schemas/</code>
2. Contract snapshots	Live vs <code>contract-snapshots/<slug>/</code>	<code>@contract-snapshot</code>
3. Mutation-proof	Broken fixtures must fail validators	<code>@mutation-proof</code>
4. DB cross-validation	Read-only SQL vs API	<code>@db + *_DB_SQL_READY</code>



Operational rules

- Update snapshots **only** with `UPDATE_CONTRACT_SNAPSHOTS=true` after intentional API change.
- Mutation suites are excluded from default Playwright runs unless `INCLUDE_MUTATION_PROOF=true`.
- DB SQL never selects password/token hashes; never writes production data.

Volume 4 — Module Architecture & Catalogue

Standard module lifecycle

1. Scaffold (`scripts/scaffold-module-hardening.mjs`)
2. Implement Api → Data → Mapper → Validator
3. Smoke + negative specs
4. Zod + contract snapshot
5. Mutation-proof
6. Real SQL when schema known → enable gate
7. npm scripts + CI module sync

Module catalogue (25)

Module	Primary MDMS area
AUTH	Login, me, devices, invitations
MASTER-DATA	Consumer/DTR/network master CRUD & lists
CONSUMERS	Profile, PQ, consumption graphs, validation
DTRS / ASSET-MANAGEMENT / FEEDER	Hierarchy, DTR analytics, feeder
HES-COMMANDS	Meter commands, job poll, history
BILLING / CONSUMPTION	Billing & consumption reports
ENERGY-AUDITS	Loss analysis, hourly loss
COMMERCIAL-ANALYSIS / TECHNICAL-ANALYSIS	Analysis report families
MIS-DASHBOARD / DASHBOARD / OVERALL-DASHBOARD	Dashboards & events
REPORTS	Event / DTR billing reports
REVENUE-PROTECTION	Aberrations, ATR, aberration entry
METER-REPLACEMENT	Submission & validation flows
NOTIFICATIONS	Mobile notifications
USERS-ADMIN / USERS-PROFILE-IMAGE	User admin
ROLE-PERMISSIONS / MODULES-PERMISSIONS	Authorization
AUDIT-LOGS	Audit APIs
UTILS-LOOKUP	Lookups & search

Depth note: All modules have pillar **scaffolding**. Deep SQL / rich mutation examples include **REVENUE-PROTECTION, ASSET-MANAGEMENT, AUTH, MASTER-DATA, UTILS-LOOKUP, METER-REPLACEMENT**. Scaffold ≠ incomplete architecture; deepen by business risk.

Volume 5 — CI/CD & Engineering Operations

Branch / promotion model

```

module-branch —PR→ QA —(post-merge gate)→ affected module suites
                        |
                        └─ green —PR→ main → full regression
  
```

Execution strategy

Command pattern	Use
<code>npm run test:module -- <slug></code>	Full module
<code>--smoke / --api / --db</code>	Scoped
<code>test:<slug>:contract</code>	Snapshots
<code>test:<slug>:mutation-proof</code>	Mutation
<code>test:hes:e2e</code>	HES async E2E

Workers default to **1** for module runners to reduce env contention (especially HES).

Reporting

- Playwright list + HTML + JSON
- Allure (`npm run report:allure`)
- Defect / finding logs for backend triage

Failure classification

Class	Example handling
Test issue	Fix assertion / data
Backend defect	Log finding; soft-path where established
Infra / HES callback	Async-pending path or soft-skip network
Contract drift	Intentional? regenerate snapshot; else fail

Volume 6 — Delivery Status, MDM Alignment & Test Accomplishments (v2.0)

This volume answers: **From a framework and test point of view, what have we done so far?**

6.1 Framework completeness (architecture)

Delivered and stable:

- Layered Api / Data / Mapper / Validator / Db / schemas / tests
- Shared ValidationEngine & AssertionEngine
- Auth global setup + token refresh
- Four quality pillars wired across ~25 modules
- Module scaffold + runners (`run-module-tests` , contract, mutation)
- Contract snapshot packs per module

- HES async poll helper (stuck detection, completion gate)
- CI module detection & GitHub Actions gates
- Enterprise documentation set (README, CONTRIBUTING, ARCHITECTURE, MODULE_GUIDE, HARDENING-STATUS, MDM coverage)

Decision: Do **not** redesign the core framework. Operate in **maintenance & expansion**.

6.2 Testing accomplishments (test POV)

Workstream	What was delivered
Functional API suites	Smoke, negative, edge, auth-negative across MDMS modules
Contract testing	Snapshot suites + regenerate workflow
Mutation-proof	Per-module proof that broken fixtures fail validators
DB cross-check	Gated harnesses; deep SQL on high-value modules
HES E2E	Init + poll; async-pending pass when callback delayed; optional strict completion
Flake control	Soft findings for known 5xx/429; HES queue gap; retries=0 on long E2E describes
Data provisioning	e.g. MASTER-DATA create-consumer assignable meter provisioning to cut flakes
Traceability	Tags (@smoke , @db , @contract-snapshot , @mutation-proof , module tags)

6.3 Alignment with [MDM Presentation_23.12.2025.pdf](#)

Question	Answer
Did we follow the PDF?	Yes — as the business catalog for API coverage
Did we automate every UI slide/chart?	No — correctly out of scope for this API framework
Module/family alignment	Strong (master, HES, MIS, commercial families, consumption, billing, RP, energy audit, ...)
Known gaps	SAIFI/SAIDI suite; dedicated Billing Exchange API (history proxy today); prepaid summary depth; some collection-report titles

Full slide-family matrix: [docs/MDM-PRESENTATION-COVERAGE.md](#)

Machine-assisted slide→endpoint map: [scripts/data/mdm-pdf-slide-api-map.mjs](#)

6.4 Maturity checklist

Maturity question	Status
New module in ~30 minutes?	Yes (scaffold + scripts)
New QA productive in one day?	Mostly — docs now support this
CI reliable every time?	Primary ops focus (env, HES, DNS)
Failures diagnosable?	Improved (findings + defect context)
Survive API evolution?	Yes via contract + mutation when process followed

6.5 What “done” means for leadership

Claim	Accurate statement
“Framework complete”	Core architecture & patterns complete
“All MDMS automated”	Major API families covered ; not every PDF chart title; SAIFI/SAIDI still a gap
“Four pillars everywhere”	Scaffolded everywhere ; deep on selected gold modules
“Green suite = healthy product”	Soft-skips / async-pending must be tracked as debt

6.6 Recommended next investments (ops, not redesign)

1. Keep pillars green in CI; manage skip debt.
2. Close **SAIFI/SAIDI** when backend APIs are available.
3. Deepen DB SQL for high-traffic modules (Consumers, Consumption, MIS).
4. Finish AUTH verify path (`mutation` / `contract` / `AUTH_DB_SQL_READY`).
5. Maintain documentation as modules evolve.

Closing statement

The Indoore Backend API Automation Framework has reached an **enterprise-ready architectural baseline**. It standardizes how MDMS backend APIs are tested, ties coverage to the official MDM presentation at the **API family** level, and provides measurable quality mechanisms (Zod, contracts, mutation, gated DB).

Future value comes from **using and maintaining** the framework — expanding module depth, reducing flakes, and supporting new product APIs — not from inventing alternate architectures.

Appendix A — Quick command reference

```
npm install
copy .env.example .env

npm run test:modules:list
npm run test:module -- <slug> --smoke
npm run test:<slug>:contract
UPDATE_CONTRACT_SNAPSHOTS=true npm run test:<slug>:contract
npm run test:<slug>:mutation-proof
<MODULE>_DB_SQL_READY=true npm run test:<slug>:db

npm run test:inventory
npm run typecheck
npm run report
npm run report:allure
```

Appendix B — Related artefacts

Artefact	Path
This handbook (source)	docs/ENTERPRISE-HANDBOOK.md
Prior PDF handbook v1.0	docs/Indoore Backend API Automation Framework - Enterprise Handbook (1).pdf
MDM product deck	MDM Presentation_23.12.2025.pdf
Slide→API map	scripts/data/mdm-pdf-slide-api-map.mjs

End of Enterprise Handbook v2.0